Cryptanalysis & Mitigations Web Application

Executive Summary

The "**Cryptanalysis & Mitigations**" web application is an interactive, minimalist educational platform designed with a clean white UI that demonstrates real-world cryptographic vulnerabilities alongside their defensive engineering solutions across three primary domains: Classical Cryptography, Asymmetric Cryptography, and Implementation Security. Using a side-by-side comparative layout, each section juxtaposes an "**Attack without mitigation**" column—where live attacks exploit weak implementations by cracking repeating Vigenère keys via Kasiski/Index of Coincidence frequency analysis, factoring small 32-bit RSA moduli in milliseconds using Pollard's Rho algorithm, and recovering secret passwords character-by-character through microsecond timing leaks in early-exit string comparisons—directly against an "**Attack with mitigation**" column, which proves how industry-standard defenses completely neutralize these threats by employing full-length random keys (One-Time Pad principle), modern 2048-bit RSA key lengths that require over 10^168 years to factor, and constant-time string comparisons (`hmac.compare_digest`) that eliminate side-channel timing signals.

Detailed Technical Architecture

1. Classical Cryptography: Vigenère Cipher

- **Concept:** Polyalphabetic substitution cipher shifting plaintext characters by a repeating alphabetical key.
- **Vulnerable Implementation (Attack without mitigation):**
  - When a short repeating key (e.g., `HACK`) is used, periodicity in the ciphertext preserves statistical properties of the language.
  - **Exploitation:** The system combines **Kasiski Examination** (calculating greatest common divisors of repeated n-gram distances) and the **Index of Coincidence (IC)** (measuring letter collision probability against standard English $IC \approx 0.066$) to determine key length. It then solves the decoupled Caesar cosets using **Chi-Squared (χ^2) frequency analysis**.
  - **Result:** Successfully recovers the secret key (or equivalent repeating keystream `HACKHACK...`) and decrypts the underlying message (`Success: True`).
- **Secure Implementation (Attack with mitigation):**
  - **Defense:** Employs the **One-Time Pad (OTP)** principle. The key is generated as an independent, uniformly random sequence strictly equal in length to the message.
  - **Result:** Index of Coincidence collapses to random noise ($IC \approx 0.038$), Kasiski finds zero structural period, and the statistical attack fails, producing nonsensical characters (`Success: False`).

---

2. Asymmetric Cryptography: Toy RSA Factoring

- **Concept:** RSA asymmetric encryption relies on the computational difficulty of factoring a composite modulus $N = p \times q$.
- **Vulnerable Implementation (Attack without mitigation):**
    - Utilizing insufficient key sizes (such as 32-bit moduli) makes N susceptible to sub-exponential and cycle-finding algorithms.
    - **Exploitation:** Executes **Pollard's Rho Algorithm**, a cycle-detection method utilizing polynomial transitions $f(x) = (x^2 + 1) \bmod N$ and GCD evaluations $\gcd(|x - y|, N)$ with an expected complexity of $O(N^{1/4})$.
    - **Result:** Factors N into primes p and q in less than 0.001 seconds, completely recovering the private key.
- **Secure Implementation (Attack with mitigation):**
    - **Defense:** Enforces industry-standard key parameters, specifically **2048-bit RSA keys** (over 600 decimal digits).
    - **Result:** Pollard's Rho algorithm times out. The application presents an empirical mathematical extrapolation showing that factoring a 2048-bit modulus requires approximately 2^{570} operations (3.82×10^{168} years), demonstrating practical computational infeasibility.

---

3. Implementation Security: Timing Side-Channel Attack

- **Concept:** Side-channel vulnerabilities demonstrate that mathematically secure algorithms can be compromised if software execution time varies based on secret data.
- **Vulnerable Implementation (Attack without mitigation):**
    - A naive string equality check terminates on the first mismatch (`early-exit`), leaking secret characters via execution time.
    - **Exploitation:** Guesses candidate characters position-by-position. Using high-precision timestamps (`time.perf_counter_ns()`), the attacker takes sample measurements and identifies the correct byte through median execution time amplification.
    - **Result:** Progressively recovers the password `SECURE42` live on screen (`Success! Secret is SECURE42`).
- **Secure Implementation (Attack with mitigation):**
    - **Defense:** Utilizes **Constant-Time Verification** (`hmac.compare_digest`), ensuring identical CPU cycles regardless of byte match positions.
    - **Result:** Eliminates the timing discrepancy. The statistical attacker is reduced to random guessing (`AAAXAAAA`), demonstrating immunity against timing analysis.

UI & Deployment Architecture

- **Interface Design:** Minimalist white aesthetic with dark typography, responsive columns, and real-time execution feedback.
- **Cloud Deployment:** Hosted via Streamlit Community Cloud, linked directly to the version-controlled GitHub repository (`Srujan1325/cryptography-project`).